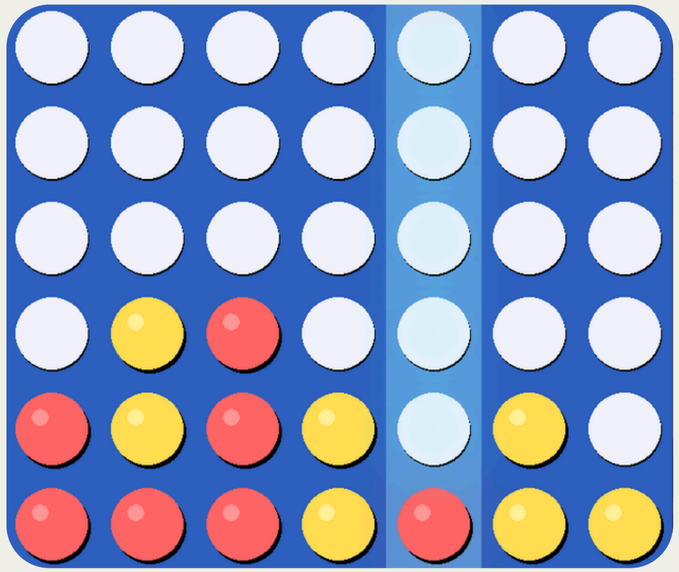
Popout-ai

MCTS AND DECISION TREES WITH ID3

José Sousa up202405046
Tiago Sousa up202405406
Duarte Gomes up202409386

Bitboard Representation: The Core Engine Behind High-Performance Game

Game Board (7*6)



O(1) Operational Efficiency

6 G	13 G	20 G	27 G	34 G	41 G	48 G
5	12	19	26	33	40	47
4	11	18	25	32	39	46
3	10	17	24	31	38	45
2	9	16	23	30	37	44
1	8	15	22	29	36	43
0	7	14	21	28	35	42

Guard Bits
Prevent overflow in columns

Bitboard Layout (7 bits per column)

mask_1 = 2^0 + 2^1 + 2^7 + 2^{14} + 2^{15} + 2^{16} + 2^{28} = 268550275
mask_2 = 2^8 + 2^9 + 2^{21} + 2^{22} + 2^{35} + 2^{36} + 2^{42} = 4501132018432

Numba and FlatNumba

Numba JIT

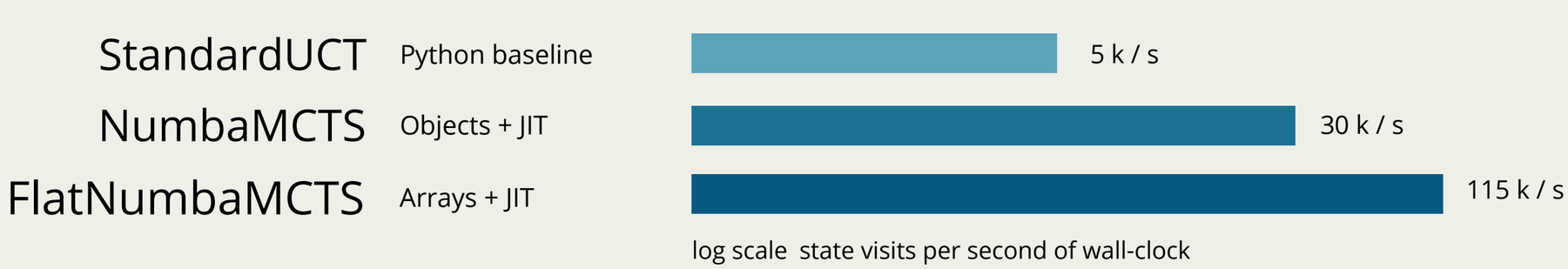
Compilação @njit · Python → LLVM

- Decorator on the four MCTS hot loops — select · expand · simulate · backprop.
- @njit(cache=True) persists compiled bytecode across runs; explicit warmup() pays the JIT cost up front.
- All operations on primitive types (int / float / numpy arrays) — no Python object access inside the kernel.
- Chosen over Cython / C extensions: same ~95% of the speed-up, zero extra build chain.

FlatNumba

Whole tree as pre-allocated NumPy arrays

- Nodes are integer indices into fixed-size arrays (visits · value · parent · children · status).
- Memory layout is contiguous → cache-friendly; no GC pressure, no Python object dispatch.
- Tree reuse: BFS compaction re-indexes the surviving subtree to slots 0..k−1 between turns.
- Visit budget compounds — each turn inherits ≈12% of the previous turn's search effort.



SPEEDUP

~ 23x

vs pure Python

MCTS Implementations

MCTS-Solver

AND/OR proof propagation - Winands et al. 2008

- Each node labelled WIN / LOSS / DRAW / UNKNOWN — labels propagate up the tree.
- WIN if ∃ child LOSS · LOSS if all children WIN + fully expanded · DRAW by consensus.
- Minimax distance breaks ties: win-fast, lose-slow.
- Search terminates instantly once the root is proven — wall-clock drops to sub-millisecond.

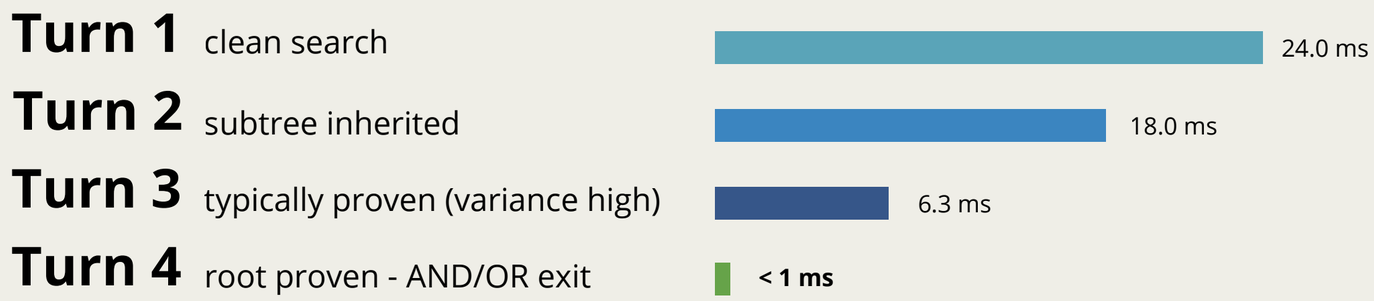
Tree Reuse

Subtree carried over between consecutive turns

- After each move, the matching child becomes the new root — the rest of the tree is discarded.
- BFS compaction re-indexes the surviving subtree into slots 0..k−1 (required by the flat-array layout).
- Solver labels survive compaction: proven nodes are not re-proved next turn.
- Each turn inherits the search effort of the previous one — depth-of-search compounds.

Wall-clock per turn - Reuse Flat Solver, 5000 iter requested

Mean of 5 trials; gains compound from reuse + proof early-exit



EFFECT

24 ms → < 1 ms

by turn 4 with the same 5000 iter/turn requested

Provably correct endgame play + deeper effective search

ID3 iris

Methodoly

Quantile discretisation + repeated k-fold CV - leakeage-safe

- Continuous features → 3 quantile bins per attribute (very_low / low / medium).
- 5 × 10-fold stratified CV → 50 independent train/test evaluations.
- Bin edges fit on training fold only — test partition never influences the splits.
- ID3 from src/decision_tree/id3/learner.py — same algorithm used downstream for PopOut.

Per-class metrics

Support = 250 samples per class - agregated over 50 folds

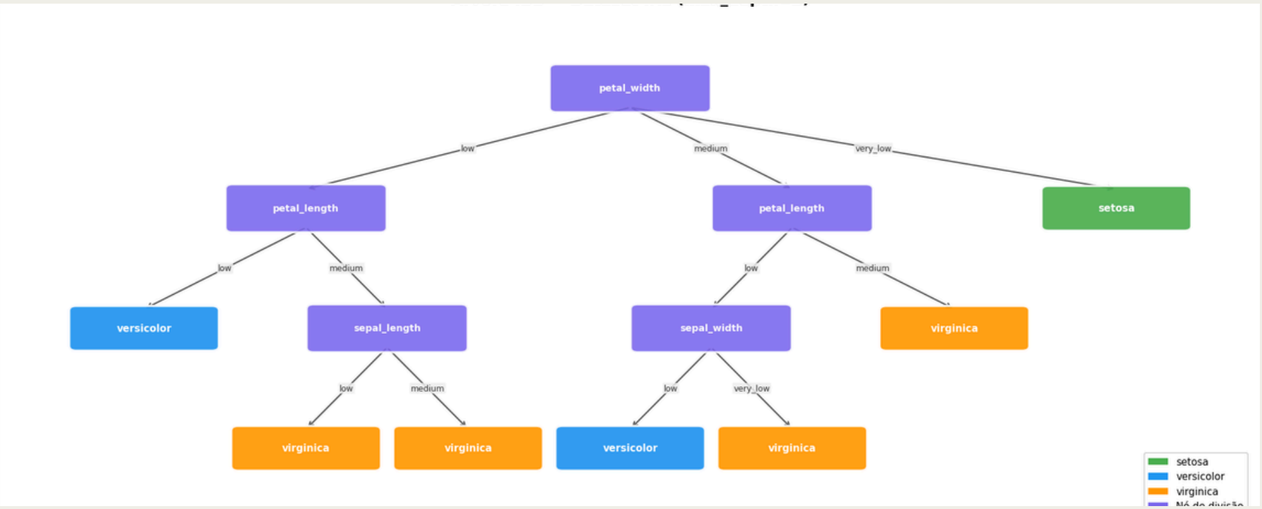
Class	Prec.	Recall	F1
Iris-setosa	0.996	0.976	**0.986**
Iris-versicolor	0.923	0.916	**0.920**
Iris-virginica	0.922	0.948	**0.935**

Overall Accuracy

94.67 %

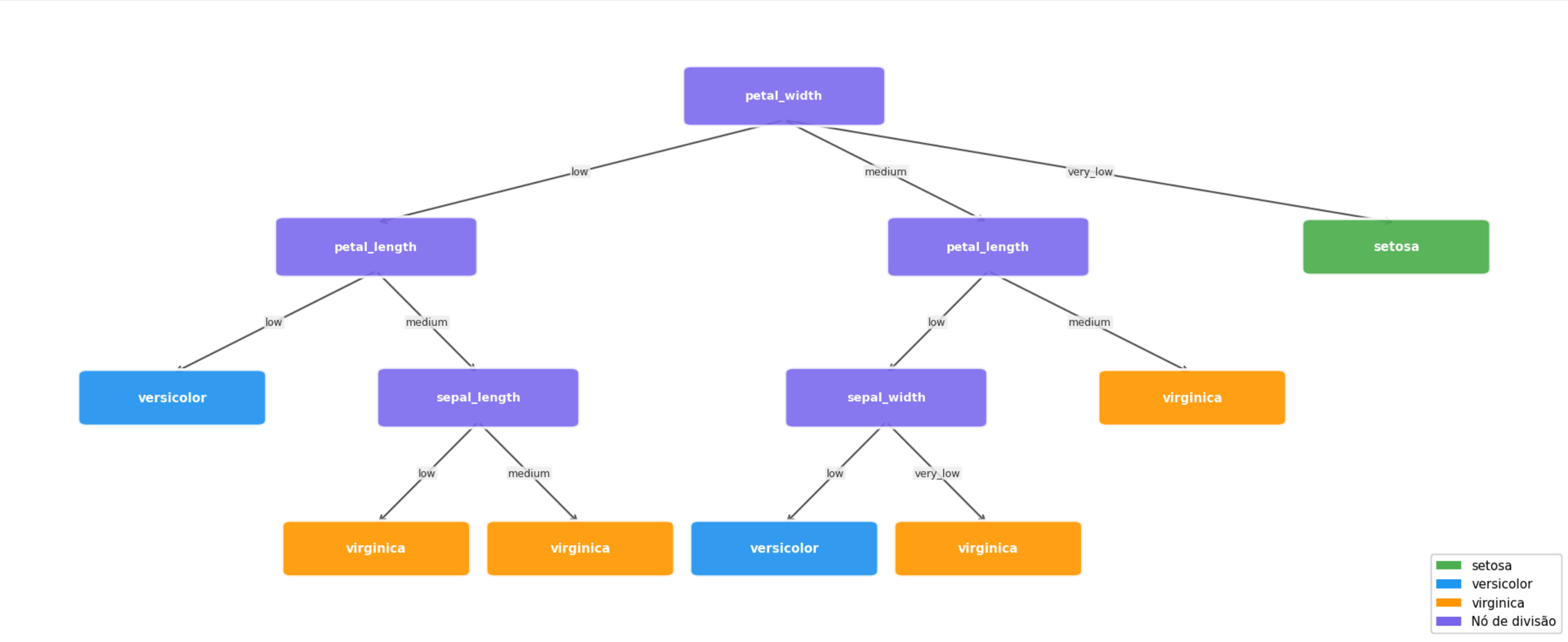
± 6.39 % (50 folds)

min 73 % · max 100 %



Same ID3Classsifer reused for PopOut downstream.

Árvore ID3- Dataset Iris



ID3 Popout AI

Dataset (v4_75k_100K)

75 000 self-play games · oracle = ReuseFlat Solver @ 100k

- 2.76 M (state, best_move) pairs from full self-play games labelled by the oracle.
- Forced 2-move openings cycling 7 × 7 = 49 column combos for coverage.
- Three blunder tiers (5 / 25 / 50 %) for diversity in opponent play.
- Horizontal mirroring doubles useful samples; 39 % of positions carry the is_proven flag.

3-split methodology

Exposed our own data leakage — reported honestly

- BUGGY: reset_index bug → 98.5 % literal row overlap (memorisation, not generalisation).
- RANDOM: correct row-level random 80/20 → 92 % overlap from intrinsic state duplication.
- GROUPED: states partitioned 80/20 before mapping back to rows → 0 % overlap, true generalisation.
- Reported all three side-by-side in the notebook.

Test accuracy across the three splits

Features model | Raw model - same trend; GROUPED is the honest measure

True Generalisation

~58 %

vs 87 % ubder leakage

Tactital accuracy on GROUPED test

opp_wins_next=1: 37 % (feat) · 39 % (raw) | pop moves: 29 % (feat) · 34 % (raw)

Hybrid disclosure

Pure ID3 vs Solver 100k: 29-36 % · hybrid: 53-54 %

Why P1 has a structural edge

Inherent first-mover tempo in Connect-4 variants

- Board opens empty — P1 claims the centre column (col 3) first, the most strategically valuable position.
- Each extra move creates new threats; P1 is always one threat ahead of P2 in the early game.
- PopOut pop mechanic: P1 can disrupt P2 defences before P2 has built a counter-structure.
- In perfectly symmetric play, the advantage compounds: P1's threats force reactive P2 play.

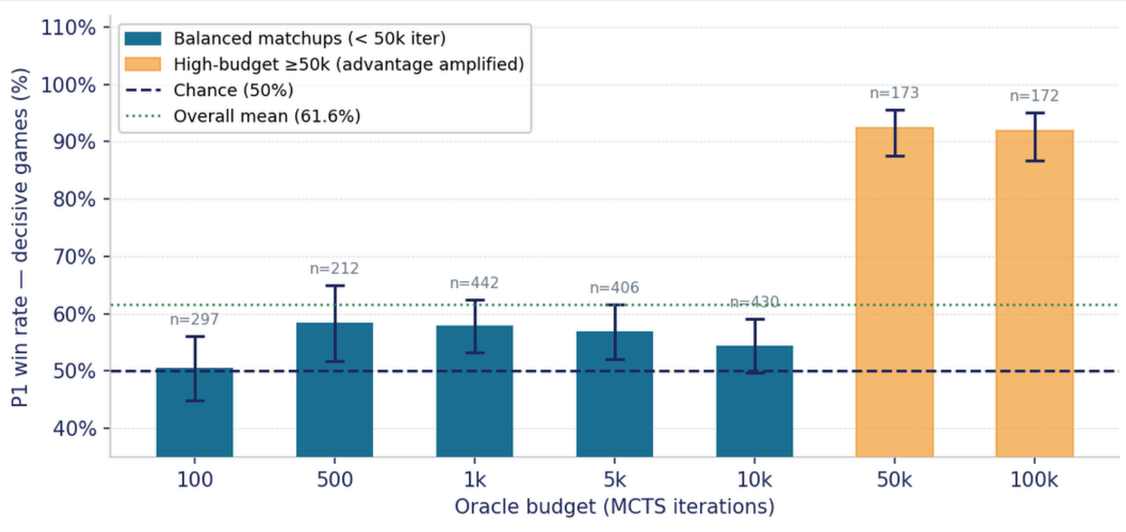
Methodology

Symmetric tournament — each agent plays 50 games per colour

- 4 000 games total across all agent types and budgets (v4_75k_100k oracle).
- Each matchup: 50 games as P1 + 50 games as P2 — colour alternation controls for agent strength.
- P1 win rate measured on DECISIVE games only (draws excluded from proportion test).
- Wilson 95 % CI and z-test used throughout — no asymptotic shortcuts for small n.

P1 win rate (decisive games) by oracle budget · 95 % Wilson CI

Bars show point estimate + CI; teal = balanced, amber = high-budget (confounded)



P1 Win Rate

61.6 %

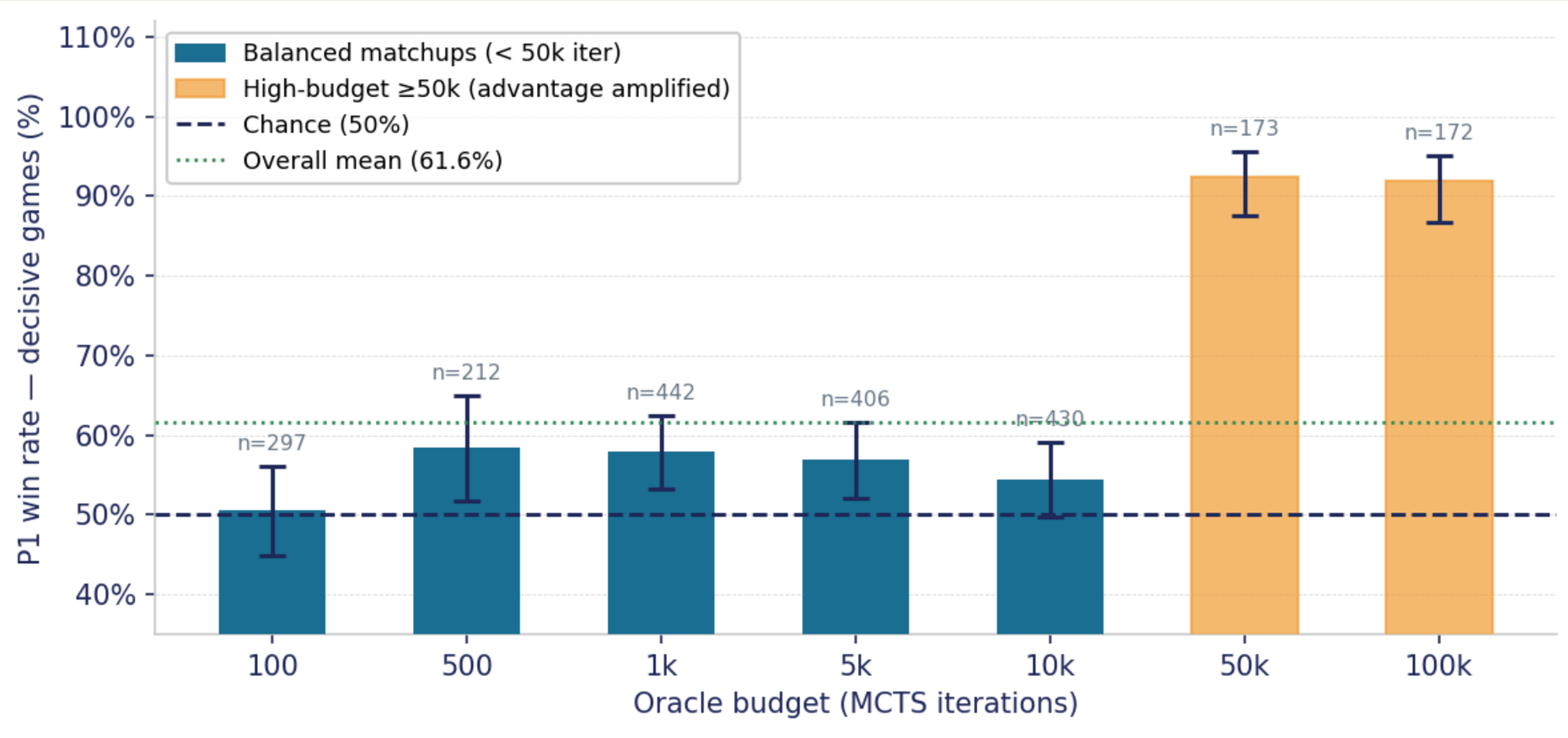
95 % CI [59.5 %, 63.6 %]

$z = 10.7$ · $p < 10^{-26}$

n = 2 132 decisive games

Draw rate 46.7 % (all games) · decisive: P1 61.6 % P2 38.4 %

P1 win rate (decisive games) by oracle budget · 95 % Wilson CI



Classification Accuracy

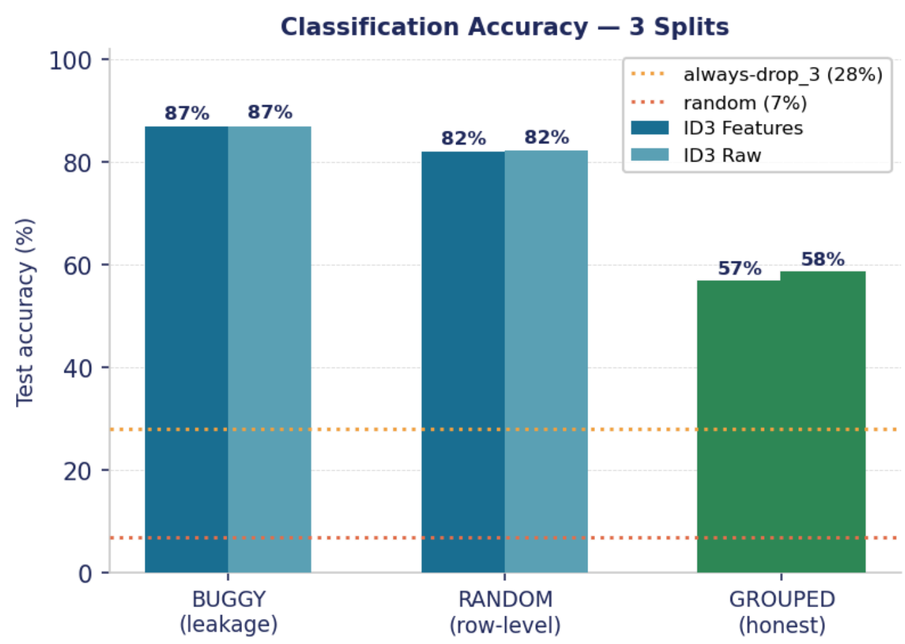
3-split methodology — leakage-safe GROUPED is the honest figure

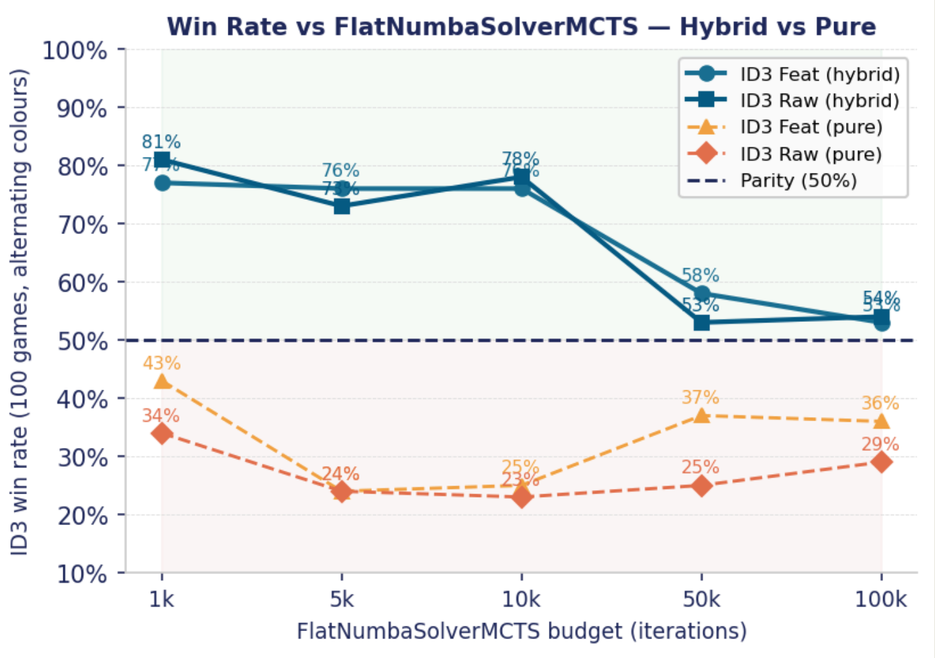
- GROUPED (0 % state overlap): 56.7 % (feat) · 58.4 % (raw).
- RANDOM (92 % overlap): 81.9 % — inflated by state duplication.
- BUGGY (98 % overlap): 86.8 % — memorisation, not learning.
- Baselines: random 7.1 % (1/14) · always-drop_3 ≈ 28 %.

Win Rate vs Oracle

FlatNumbaSolverMCTS · 100 games, alternating colours

- Hybrid @ 100k iter: 53 % (feat) · 54 % (raw) — at parity.
- Hybrid @ 1k–10k iter: 73–81 % — dominates weaker oracle.
- Pure tree @ 100k iter: 29–36 % — without tactical safeguards.
- Safeguards add +17–40 pp: most value at high oracle budgets.





GROUPED

58 %

test accuracy (raw)

HYBRID WR

53 - 54 %

vs oracle @ 100k iter

GROUPED 8× better than random baseline · 2× better than always-drop_3

MCTS SPEEDUP

~ 23x

FlatNumba vs pure Python

GROUPED ACC

~ 58 %

ID3 — honest generalisation

HYBRID WIN RATEGROUPED

53 - 54 %

vs oracle @ 100k iter

P1 WIN RATE

61.6 %

decisive games (n = 2132)

MCTS Engine

FlatNumba · Solver · Tree Reuse · TopK-UCT

- FlatNumba kernel: ~40× over pure Python — 100 k iter in real time.
- MCTS-Solver: AND/OR proof propagation, provably correct endgame.
- Tree reuse: 24 ms (turn 1) → < 1 ms (turn 4) — same 5 k budget.
- TopK-UCT: k ≥ 7 as stable as k = 14; 50 % selection cost saved.

ID3 Decision Tree

Built from scratch · 3-split leakage audit

- 2.76 M positions from 75 k self-play games labelled by oracle.
- GROUPED accuracy 58 % — 8× random baseline, 2× always-drop_3.
- 3-split audit revealed own data-leakage bug (87 % was memorisation).
- Tactical sub-tasks harder: 37–39 % (opp_wins_next) · 29–34 % (pop).

Game Performance

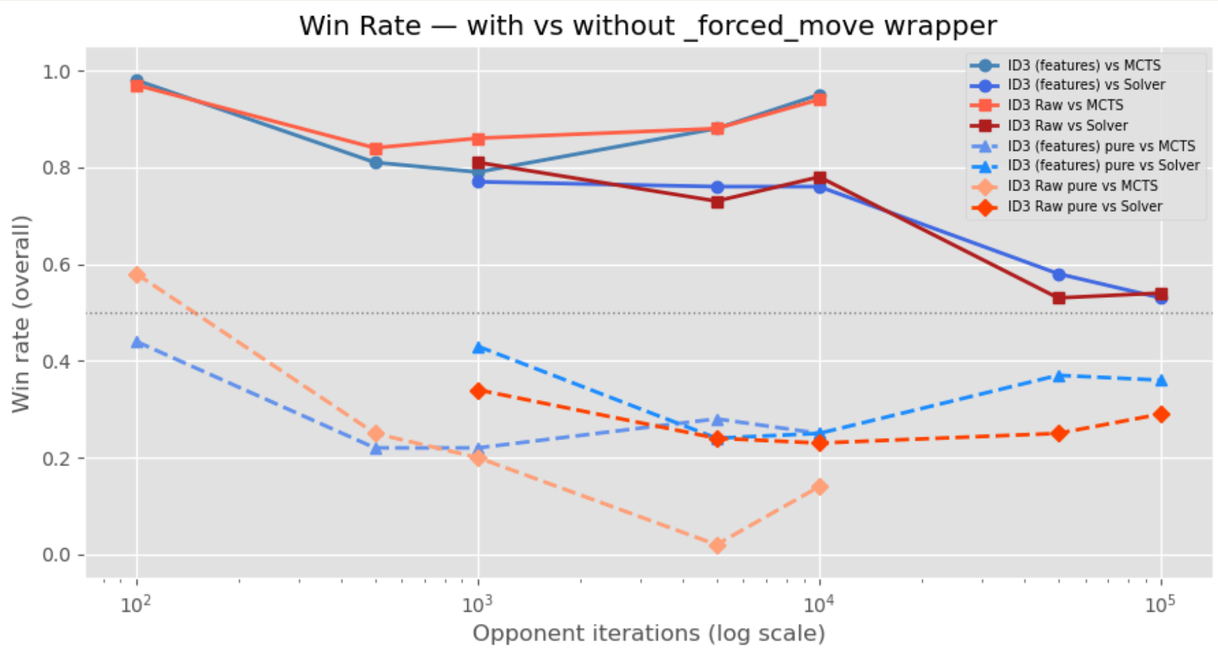
100 games/matchup · alternating colours · Wilson CI

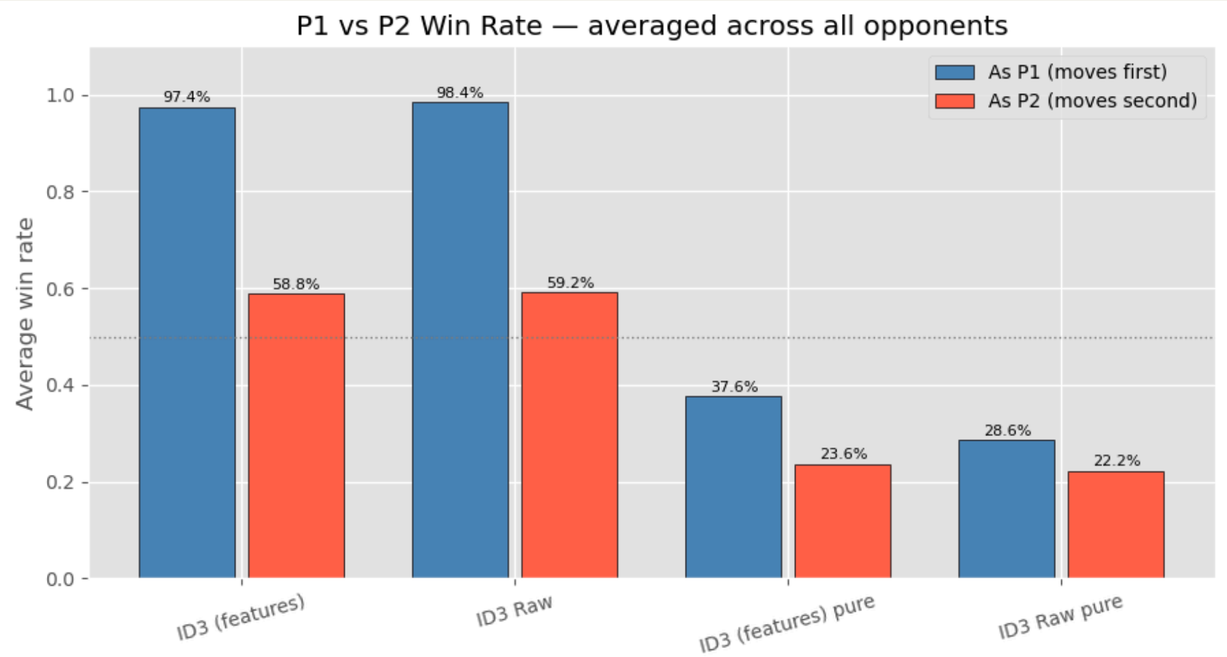
- Hybrid vs oracle @ 100 k: 53–54 % — at statistical parity.
- Hybrid vs oracle @ 1 k–10 k: 73–81 % — decisively dominates.
- Tactical safeguards add +17–40 pp over pure ID3 tree.
- P1 advantage: 61.6 % decisive games ($z = 10.7$, $p < 10^{-26}$).

Key Takeaway

A from-scratch ID3 tree trained on oracle self-play reaches parity with that same oracle when equipped with tactical safeguards — proving that well-structured imitation learning can match near-perfect search at the same compute budget.

Win Rate Results





Effect of Tatical Safeguards (_forced_move)

- Hybrid = tree search + forced-move safeguards (win/block immediate threats).
- Pure = tree search only — no tactical override.
- At 100 k oracle: hybrid 53–54 % vs pure 29–36 % → safeguards add +17–40 pp.
- At 1 k–10 k oracle: hybrid 73–81 % — dominates as oracle is weaker.
- The safeguard benefit is largest at high oracle budget where tree alone falls short.

HYBRID @ 100k ORACLE

53 - 54 %

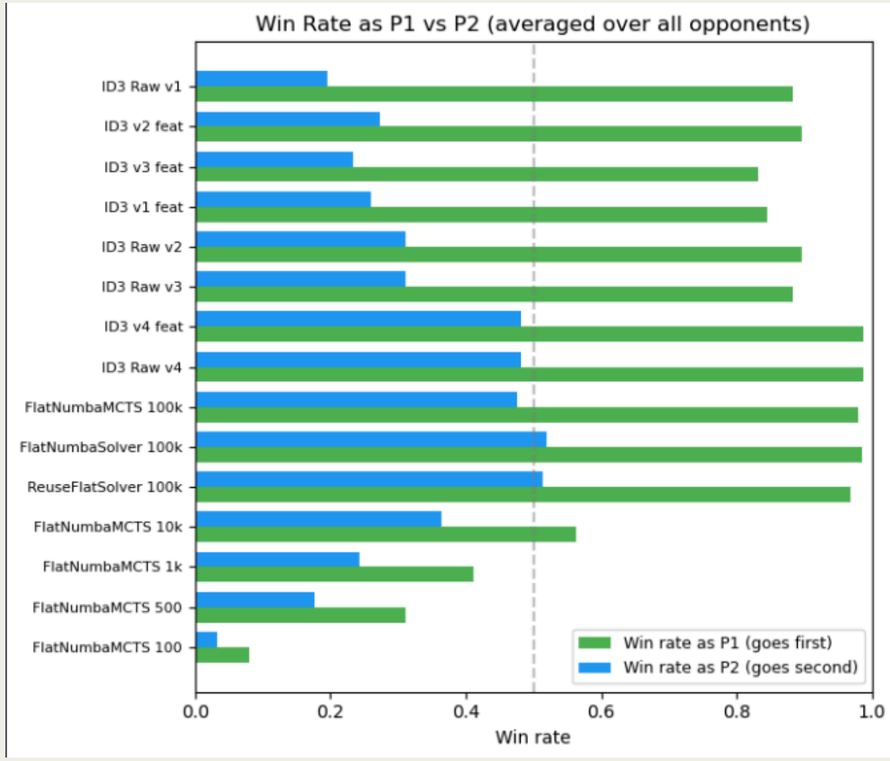
at parity with oracle

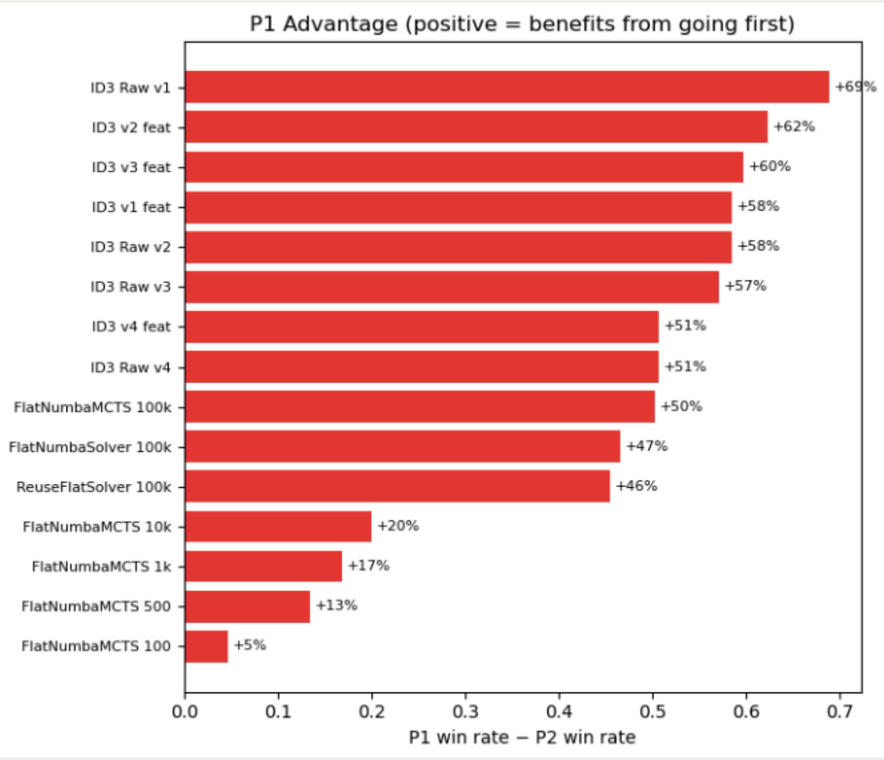
HYBRID @ 100k ORACLE

29 - 36 %

safeguards add +17-40 pp

Tournament Summary





P1 WIN RATE

61.6 %

of decisive games

ID3 ADVANTAGE

+ 51% to +69%

vs MCTS +13% to +50%

$z = 10.7 \cdot p < 10^{-26}$

n = 2 132 decisive games

ID3 P1 Advantage by Version

- v1–v3 ID3: P1 ≈ 82–91 %, P2 ≈ 15–25 % — advantage large but agent is weak overall.
- v4 hybrid: P1 ≈ 97–98 %, P2 ≈ 59 % — strongest AND most balanced agent.
- P2 win rate rises with agent quality — better agent compensates for going second.
- P1 advantage is structural: present even in pure MCTS agents.